

Flood Digital Twin — Team Work Breakdown

Three technical tracks, the tools each one uses, and how it all fits together

Purpose of this document: each of your three teammates can read just their own section and know exactly what to build and what tools to use. The tool explanations are written so a non-technical reader (a judge, a teammate on a different track, a mentor) can understand what each piece of technology actually does for the product — not just its name.

How the Work Is Split

Track	Owns
Physics Engine	The flood simulation itself, the shared "clock" that keeps everything in sync, and turning raw simulation output into map-ready images
RAG + Human/Landscape Data	Turning historical flood reports and map features into structured knowledge, the AI critique system, the data agreements between tracks, and translating the flood simulation into plain facts (impact extractor, search-area tool)
Frontend (UI/UX)	Everything the trainee actually sees and clicks on — the 3D map, the training mode screens, the timeline

Two pieces — the shared memory of "what's happening right now" (session state) and an optional chat feature (post-launch, not needed for the demo) — are jointly owned and explained at the end.

1. Physics Engine

In plain terms, this group's job is: take real government flood-prediction data and turn it into an accurate, moment-by-moment picture of how deep and how fast the water is at every point along the river, fast enough to drive a live 3D map.

Tools this group uses

NumPy

A widely-used Python library for doing fast math across huge grids of numbers all at once, instead of one calculation at a time. What it does for the product: this is the workhorse that lets the engine calculate flood depth across an entire map, thousands of points at once, quickly enough to keep up with a live simulation.

rasterio

A library for reading and writing geographic image files — think of it like a photograph of the land where every pixel secretly holds a real elevation or depth value, instead of just a color. What it does for the product: lets the engine read the terrain's elevation data and write out flood-depth "images" in a format that mapping software can display.

GDAL

The underlying translator that rasterio and almost all mapping software is built on top of — it converts between the many different technical formats and coordinate systems that different data sources use. What it does for the product: makes sure terrain data from one government agency, river-flow data from another, and map data from a third all line up correctly on the same map, even though none of them were originally built to work together.

xarray

A tool for organizing data that changes over time and space — like a stack of flood-depth snapshots, one per minute, each covering the whole county. What it does for the product: keeps the simulation's time-based data organized so the engine can quickly answer "what did the flood look like at 2:30 a.m.?" without re-processing everything from scratch.

PyTorch (used narrowly, stretch goal only)

The same industry-standard machine learning toolkit used by most AI companies — here it's used for one small, optional job, not the main flood prediction. What it does for the product: if time allows, it lets the system apply a learned correction to fix a known blind spot in the government flood method (it tends to underpredict on smaller creeks) — but the system fully works and is fully demoable with this switched off.

TiTiler / rio-tiler

Tools that chop one giant map image into small square tiles — the same trick Google Maps uses so you're not loading a picture of the entire planet every time you scroll. What it does for the product: lets the 3D map load smoothly and quickly, fetching only the small piece of the flood map currently visible on screen instead of the whole county at once.

cesium-terrain-builder

Converts very detailed elevation scan data (from an aircraft-mounted laser survey called lidar) into a format the 3D map software can display efficiently as realistic hills and valleys. What it does for the product: gives the 3D map real, accurate ground shape for the water to appear to flow over, instead of a flat, generic surface.

FastAPI (powering the clock service)

A lightweight tool for building small, fast web services. What it does for the product: powers the shared "clock" that keeps every other part of the system — the flood simulation, the AI critique, and the map — agreed on exactly what moment in the timeline they're all looking at, so nothing ever shows contradictory information.

WebSocket

A technology that lets a server push updates to a webpage the instant something changes, instead of the webpage having to keep asking "is there anything new yet?" What it does for the product: lets the clock's ticks reach the map and timeline instantly, so the simulation feels live rather than laggy.

What this group builds, in order

- 1 The clock service (small, but everything else depends on it existing first). **Tool: FastAPI**, with **WebSocket** to push updates out.
 - 2 Hydrology data ingestion — pulling in real river-gauge and rainfall-based streamflow data. **Tools: GDAL** (format/coordinate translation) feeding into **NumPy** and **xarray** (organizing the incoming data).
 - 3 Terrain ingestion — converting the detailed elevation scan into map-ready 3D terrain. **Tool: cesium-terrain-builder**.
 - 4 The physics engine itself — turning streamflow into depth and speed of water, for every point on the map, at every moment in the timeline. **Tools: NumPy** (the core math), **rasterio** and **GDAL** (reading/writing the geographic data), **xarray** (organizing it by time).
 - 5 Tiling the output so the map can display it quickly. **Tools: TiTiler / rio-tiler**.
 - 6 (Stretch) The optional correction step. **Tool: PyTorch**.
-

2. RAG + Human/Landscape Data

In plain terms, this group's job is: take everything the county already knows — from past flood reports to the exact locations of every road and camp — and turn it into something an AI can search through and reason about, and then build the AI system that critiques a trainee's decisions using that knowledge.

Tools this group uses

sentence-transformers (or an API embedding model)

A tool that converts a piece of text into a list of numbers that captures its *meaning*, so a computer can tell how similar two passages are even if they don't share any of the same words. What it does for the product: lets the system find the historical flood report most relevant to whatever a trainee just proposed, even if the trainee's wording has nothing in common with the

original report's wording.

pgvector / FAISS

Specialized search tools built to instantly find the "closest matches" among a large collection of those meaning-numbers. What it does for the product: makes searching through every past flood report practically instant, so the AI critique never feels slow, even as the library of historical incidents grows.

PostGIS

An add-on to a standard database that understands geography — shapes, distances, and boundaries — not just plain numbers and text. What it does for the product: stores the exact location of every road, low-water crossing, building, and camp along the river, so the system always knows precisely what's where on the real map.

geopandas / shapely / rasterstats

Python tools for asking geographic questions like "which buildings fall inside the flooded area right now?" or "what's the water depth on top of this specific road?" What it does for the product: this is the actual machinery behind translating a flood map into plain facts — it's what lets the system say "this specific crossing is impassable" instead of just showing a vague blue shape on the map.

PyTorch (historical-decision scorer, stretch goal only)

The same toolkit mentioned above, used here — if time allows — to attach a rough numerical score to a proposed tactic based on how similar tactics performed historically. What it does for the product: adds an extra, clearly-labeled hint alongside the AI's written critique — it never replaces the written, cited explanation, only supplements it.

FastAPI (running the AI critique system)

The same lightweight web-service tool used by the Physics Engine group, here powering the "brain" of the product. What it does for the product: runs the logic that reads the current flood facts and the historical reports, and decides what feedback to give a trainee at each decision point.

What this group builds, in order

- 1 Lead the upfront agreement session on exactly what data gets passed between all three groups (so nobody builds against guesses). No specific tool — this is a planning task.
 - 2 Load in the map data — every road, crossing, building, and camp, each with a precise location. **Tool: PostGIS.**
 - 3 Build the translator that turns the Physics Engine's raw flood output into plain facts ("this road is now underwater"). **Tools: geopandas, shapely, and rasterstats**, reading directly from Physics Engine's raster output and PostGIS's map data.
 - 4 Digest the historical flood reports into a searchable library. **Tools: sentence-transformers** (or an API embedding model) to convert the reports into searchable meaning, **pgvector or FAISS** to store and search them quickly.
 - 5 Build the AI system that reads a trainee's proposed plan and responds with cited, historically-grounded feedback. **Tool: FastAPI**, running the AI logic and calling on the sentence-transformers/pgvector search from step 4.
 - 6 Build the safety check that rejects any AI response that references something that doesn't actually exist in the historical library or the current flood facts. Built as a rule-checking layer inside the same **FastAPI** service from step 5 — no separate new tool.
 - 7 Build the tool that estimates a probable search area for a missing person, based on water speed and direction — deliberately shown as a shaded zone of probability, never a single confident point, since a fast-moving flood makes a single-point guess both unreliable and irresponsible to present as fact. **Tools: geopandas and shapely** (the same geometry toolkit as step 3), reading the velocity data directly from Physics Engine.
 - 8 (Stretch) The optional numerical tactic-scoring model. **Tool: PyTorch.**
 - 9 (Post-launch, not needed for the demo) An open-ended chat feature, in addition to the structured training-mode critique. **Tool: FastAPI**, extending the same AI logic from step 5.
-

3. Frontend (UI/UX)

In plain terms, this group's job is: build the actual screen a trainee looks at and interacts with — a navigable 3D scene of the county with the flood and the AI's feedback laid on top, synced to a timeline they can scrub back and forth through.

Tools this group uses

TypeScript

A version of JavaScript (the programming language almost every website is built in) with extra built-in safety checks that catch mistakes before they become bugs. What it does for the product: this is the language the entire visible app — map, buttons, timeline, training-mode screens — is written in, and the safety checks help avoid last-minute bugs showing up during a live demo.

React + Next.js

The most widely used framework for building interactive web applications, plus a companion tool that helps organize and quickly serve that application to a browser. What it does for the product: this is the toolkit used to build every screen and interactive element a trainee sees and clicks through.

CesiumJS (preferred)

A specialized library for rendering realistic, navigable 3D globes and terrain inside a web browser — the same category of technology used by organizations like NASA and the US Air Force for geographic visualization. What it does for the product: this is what actually draws the realistic 3D hills, valleys, and rising floodwater that a trainee or judge sees and can rotate and zoom around.

deck.gl (fallback option)

A faster, simpler library for drawing large amounts of map data — less visually realistic than CesiumJS, but much quicker to get running. What it does for the product: a backup plan — if the fully realistic 3D terrain proves too time-consuming to finish before the deadline, this still delivers an impressive, fully functional map with much less setup time.

WebSocket (receiving end, shared concept with Physics Engine)

The same real-time update technology described in the Physics Engine section, used here to receive those updates. What it does for the product: lets the map and timeline update themselves the instant the simulated clock ticks forward, with no manual refreshing needed.

What this group builds, in order

- 1 Load the static 3D terrain onto the map — this can start immediately, using placeholder data, without waiting on the other two groups. **Tools:** **CesiumJS** (or **deck.gl** as the faster fallback), built inside **React + Next.js** and written in **TypeScript**.
- 2 Build the timeline scrubber, wired to the shared clock. **Tool:** **WebSocket**, receiving updates from Physics Engine's clock service.
- 3 Layer the live flood-depth image on top of the terrain. **Tool:** **CesiumJS/deck.gl**, displaying the tiles Physics Engine produced with TiTiler/rio-tiler.
- 4 Layer the AI system's markers and shaded zones on top of that. **Tool:** **CesiumJS/deck.gl**, displaying the overlay data RAG + Data's AI system produced.
- 5 Build training mode — the screens where a trainee enters their plan at each decision point and sees the AI's cited response. **Tools:** **React + Next.js**, in **TypeScript**.
- 6 (Post-launch, not needed for the demo) The chat tab's visible interface. **Tools:** **React + Next.js**, calling RAG + Data's FastAPI chat logic.

What's Shared Across All Three Groups

Session state (the system's shared short-term memory). This is the one place where data from all three groups meets — the current time, the current flood facts, the trainee's plan, and the AI's responses all live here so that the map, the training-mode screen, and (later) the chat feature never show three different, contradictory versions of what's happening. No single group's information passes through it exclusively, so its exact shape needs sign-off from all three before anyone builds against it, even though the Frontend group is the natural choice to actually build and maintain it, since they read from it the most.

Chat tab. An open-ended conversational feature, explicitly the lowest priority in the entire project — the structured training-mode experience already delivers the core value (a cited, historically-grounded critique) without it. If there's spare time at the very end, it's a joint effort between the Frontend group (the visible chat window) and the RAG + Data group (the logic answering the questions).

How It All Connects — A Non-Technical Walkthrough

Imagine the trainee has just started the Kerr County scenario. Here's what's actually happening, in plain language, at each step:

- 1 **A real government river gauge reports rising water.** The Physics Engine group's system picks this up and, using the same kind of flood-prediction method the National Weather Service itself uses, calculates how deep and how fast the water is at every point along the river, for this exact moment in the timeline.
- 2 **That raw flood data gets translated into plain facts.** The RAG + Data group's translator looks at the flood data alongside the map's real roads, crossings, and camps, and produces a simple list: "this crossing is now impassable," "this camp's exit road floods in 22 minutes." Nothing downstream ever has to understand raw flood-simulation numbers — just this plain list.
- 3 **The trainee is shown a decision point** (matching the real times from the actual 2025 event — 1:14 a.m., 2:30 a.m., 4:03 a.m.) and types in what they'd do.
- 4 **The AI system checks their plan against history.** It searches the library of real past flood reports for the most similar situations, and responds with specific, cited feedback — "in the 2015 Wimberley flood, a similar delay in issuing an alert led to X outcome" — never a vague or made-up-sounding claim, because a safety check actively blocks any AI response that isn't backed by something real in the historical library or the current flood facts.
- 5 **The 3D map shows all of this at once** — the terrain, the rising water, and markers for what the AI just flagged — all keeping perfect time with the timeline the trainee is scrubbing through, because everything reads from the same shared "clock" and shared "memory" of what's currently happening.
- 6 **At the final decision point,** if the scenario calls for a search-and-rescue decision, the system shows a shaded area of probability for where a missing person might be — based on water speed and direction — rather than a falsely confident single point, because that's the responsible way to present this kind of uncertain, high-stakes information.

That's the whole product: real government flood science, real historical accountability, and a live, explorable 3D view of both — built by three teams working on three different, clearly bounded layers that meet at a small number of well-defined handoff points.